FLUTTER MULTIPLATFORM ANDROID & WEB DOKUMENTASI TEKNIS

Panduan Lengkap Mengaktifkan dan Build Flutter Web (Multiplatform Android & Web)

Dokumen ini berisi panduan langkah demi langkah (*step-by-step*) untuk mengaktifkan dukungan Flutter Web pada proyek Flutter yang sudah ada, menangani logika kode lintas platform agar build Android tidak rusak, serta proses build dan distribusi web dalam bentuk ZIP.

---

1. Mengaktifkan Platform Web pada Proyek yang Sudah Ada

Secara default, jika proyek dibuat tanpa platform web atau platform web sempat dihapus, Anda dapat mengaktifkannya kembali menggunakan Flutter CLI:

Opsi A: Mengaktifkan Web saja

```bash
flutter create --platforms=web .
```

Opsi B: Mengunci proyek hanya untuk Android dan Web (Rekomendasi)

Jika ingin membatasi proyek agar tidak menyertakan boilerplate desktop/iOS yang tidak terpakai:

```bash
flutter create --platforms=android,web .
```

> 💡 **Catatan:** Perintah di atas akan:
> - Membuat folder `web/` lengkap dengan `index.html`, `manifest.json`, favicon, dan icon.
> - Memperbarui file konfigurasi internal `.metadata` pada baris `migration.platforms`.

---

2. Penyesuaian Logika Kode Dart (`kIsWeb`)

Beberapa package (seperti `webview_flutter`, `path_provider`, atau penggunaan `dart:io`) menggunakan channel native Android/iOS yang tidak didukung atau perilakunya berbeda di browser. Jika langsung dipanggil di Web, aplikasi bisa *freeze*, melempar error, atau blank.

Gunakan `kIsWeb` dari `foundation.dart`

Import konstanta `kIsWeb`:

```dart
import 'package:flutter/foundation.dart';
```

Contoh proteksi controller native (seperti pada `profile_screen.dart`):

DART

```dart
// 1. Pada initState: Cegah inisialisasi controller native di browser
@override
void initState() {
  super.initState();
  if (kIsWeb) {
    _isLoading = false;
    return; // Controller native hanya dibuat di platform mobile
  }

  try {
    _controller = WebViewController()
      ..setJavaScriptMode(JavaScriptMode.unrestricted)
      ..loadRequest(Uri.parse('https://www.google.com'));
  } catch (_) {
    _isLoading = false;
  }
}

// 2. Pada Widget build(): Berikan tampilan UI fallback di Web
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Google')),
    body: Stack(
      children: [
        if (kIsWeb)
          const Center(
            child: Text(
              'In-app WebView aktif di Android & iOS.\n'
              'Untuk versi Web, buka https://www.google.com secara langsung.',
              textAlign: TextAlign.center,
            ),
          )
        else if (_controller != null)
          WebViewWidget(controller: _controller!)
        else
          const Center(child: Text('https://www.google.com')),
      ],
    ),
  );
}
```

> 💡 **Aturan Umum:**
> - Hindari `import 'dart:io';` untuk mengecek platform (`Platform.isAndroid`). Gunakan `Theme.of(context).platform` atau `defaultTargetPlatform` jika butuh cek platform OS tanpa merusak Web.
> - `kIsWeb` bernilai `true` saat di browser dan `false` saat di Android/iOS/desktop.

---

3. Mengambil Dependensi Terbaru

Setelah platform aktif dan kode disesuaikan:

```bash
flutter pub get
```

Verifikasi kualitas kode:

```bash
flutter analyze
flutter test
```

---

4. Build Flutter Web (Production Release)

Command Standar

```bash
flutter build web --release
```

Output hasil kompilasi HTML, JavaScript, dan WebAssembly akan berada di folder `build/web/`.

Parameter Tambahan: `--base-href`

Jika aplikasi akan di-deploy ke subdirektori (misalnya GitHub Pages: `https://username.github.io/nama-repo/`):

```bash
flutter build web --release --base-href "/nama-repo/"
```

(Untuk root domain atau localhost, parameter ini tidak diperlukan).

---

5. Mengompres Hasil Build Web ke File ZIP

Untuk membagikan atau mengunggah build web ke server:

```bash
(cd build/web && zip -r ../../smart_prototipe_web.zip .)
```

Verifikasi ukuran file zip yang terbentuk di root proyek:

```bash
ls -lh smart_prototipe_web.zip
```

---

6. Mengapa Build Web Tidak Bisa Dibuka Langsung (`file://`)?

Jika file `build/web/index.html` diklik ganda (*double-click*), browser akan menampilkan halaman kosong (**blank**). Hal ini normal karena:

1. **CORS & Fetch Policy:** Browser melarang pembacaan aset (`main.dart.js`, `flutter_bootstrap.js`) via protokol `file://`.

2. **WebAssembly (WASM):** Streaming kompilasi CanvasKit WASM diblokir di luar konteks web server.

3. **Base Href:** `` pada `file://` akan merujuk ke root hard drive sistem operasi, bukan ke folder aplikasi.

Cara Menjalankan Hasil Build di Komputer Lokal

Gunakan salah satu cara berikut untuk menjalankan web server lokal:

Opsi 1: Python HTTP Server (Bawaan Mac / Linux)

```bash
cd build/web && python3 -m http.server 8080
```

Buka browser: **`http://localhost:8080`**

Opsi 2: Node.js `npx serve`

```bash
npx serve build/web
```

Opsi 3: Menjalankan Langsung via Flutter CLI (Development)

```bash
flutter run -d chrome
```

7. Build Android APK (Release)

Untuk memastikan build Android tetap berjalan sempurna berdampingan dengan platform Web:

```bash
flutter build apk --release
```

Output APK release:

`build/app/outputs/flutter-apk/app-release.apk`

Ringkasan Rantai Perintah (Cheat Sheet)

```bash
# 1. Aktifkan platform web
flutter create --platforms=android,web .

# 2. Update dependensi & cek kode
flutter pub get
flutter analyze
flutter test

# 3. Build Web & ZIP
flutter build web --release
(cd build/web && zip -r ../../smart_prototipe_web.zip .)

# 4. Build Android APK
flutter build apk --release

# 5. Tes Web di Browser Lokal
cd build/web && python3 -m http.server 8080
```